

Title	AXI 64bit-to-32bit Converter Design Note
Project	AXI
Description	AXI 64bit-to-32bit Converter 설계 방법
Author	holelee
Date	31/Jul/2005

(*) AXI 64bit-to-32bit Converter

SkyWalker 프로젝트는 32bit AXI Interconnect를 사용한다는 Spec이 결정이 되었다. 칩 내부의 대부분의 logic이 32bit AXI interface로 연결되지만, 단 하나의 IP가 64bit AXI interface를 사용하게 되는데, 그것이 Graphic Accelerator이다. Graphic Accelerator는 추후에 보다 높은 bandwidth를 사용하는 application의 경우 64bit AXI interconnect에 직접 연결할 수 있도록 우선은 SkyWalker 프로젝트에서는 over spec으로 설계된 것이다. 64Bit AXI interface의 Graphic Accelerator를 32bit AXI interconnect에 연결할 수 있는 방법이 필요한데, converter를 설계하여 다는 방법을 사용하기로 했다. 이를 위해 설계되는 logic이 64bit-to-32bit Converter이다.

Graphic Accelerator는 64bit AXI interface spec의 모든 feature를 사용하는 것이 아니므로, 우선은 Graphic Accelerator를 연결할 수 있는 제한된(그렇지만 size가 작은) 64bit-to-32bit converter를 설계하고, 추후에 모든 AXI spec을 만족하는 converter를 설계하도록 한다.

(*) 동작

복잡한 것은 모두 생략하는 것이 편하므로, 우선은 INCR burst만 지원하고 unaligned burst는 지원하지 않는다고 가정한다.

- Read의 동작

1. Read Request 처리

- 기본적으로 64bit interface에서 들어오는 request를 32bit interface에서 모두 accept하기 전까지는 새로운 64bit interface request를 accept하지 않도록 한다. 이런 처리를 하더라도 어쨌든 32bit interface가 받을 수 있는 만큼 계속 request가 전달되므로 성능의 저하는 없다.
- 64bit interface에서 들어오는 request가 64bit request인 경우 Burst Length가 8 이상이면 두 개의 32bit request로 split하여 전송한다.
- 64bit interface에서 들어오는 request가 64bit request인 경우 Burst Length가 8 이하이면 1 개의 32bit request로 전송된다.
- 64bit interface에서 들어오는 request가 32bit 이하의 request인 경우, 그 대로 32bit interface로 전달된다.

2. RDATA Channel 처리

- 64bit request에 대한 data의 경우 binary flag에 맞추어 32bit data 두 개를 64bit 으로 합치는 일을 수행한다.

- 32bit 이하의 request에 대한 data의 경우 binary flag에 맞추어 32bit RDATA32를 64bit RDATA64의 상위 32bit 혹은 하위 32bit으로 mapping해야 하는데 굳이 그럴 필요가 없어 보인다. RDATA64 = {RDATA32, RDATA32}로 전달해도 무방하다.
- RLAST 신호를 생성하여 64bit interface로 전달해야 한다.
- 이런 처리를 위해서 다음과 같은 정보가 필요하다.
 - RLAST 신호 생성을 위해 Request된 Burst Length : 64bit interface로 전달된 RVALID & RREADY를 count하여 RLAST를 생성한다.
 - 아니면 32bit interface에서 들어오는 RLAST를 count하여 보낼 수도 있다. 이 경우에는 request가 split되었는지만 detect하면 된다.
 - request가 64bit였는지 아니면 32bit이하였는지 detect할 필요가 있다.
 - RID for out-of-order completion
- 위의 필요한 정보들은 request마다 저장되어야 하고 그 저장되는 공간은 여러개 이어야 한다.. 즉 한번에 여러개의 64bit read request를 처리할 수 있어야 한다. SDRAM controller의 경우 read request가 먼저 SDRAM Controller에 동작하는 것이 좋으므로, RDATA processing이 모두 끝난 다음에 새로운 read request를 받으면 좋지 않다. 이 때 저장되는 공간은 que(FIFO)가 되면 안된다. Out-of-order complete이 존재하므로 que가 되면 안되고 random access가 가능한 공간이어야 한다. RID를 이용하여 lookup 할 수 있으면 좋다. 단 ARID를 사용하지 않는다면 que 형태도 가능하다.
- 64bit request 처리에 있어서 RDATA32를 RDATA64로 만들때, 첫번째 들어오는 RDATA32를 저장할 register는 하나만 존재하면 안된다. 동시에 처리 가능한 request의 숫자나 RID의 숫자만큼 존재해야 한다. Out-of-order completion 때문에 RDATA32이 마구 섞여 들어올 수 있기 때문이다.
- Write의 동작
 1. Write Request 처리
 - Write Request의 처리는 Read Request의 처리와 동일하다.
 2. WDATA Channel 처리
 - 64 bit request의 경우 WDATA64를 2 cycle동안 WDATA32로 나누어 보내야 한다.WSTRB도 마찬가지로 처리된다.
 - 32 bit 이하의 request의 경우 WDATA64중 address에 맞추어 WDATA32로 보내져야 한다. 이때는 AWSIZE도 고려되어야 한다. 예를 들어 byte의 경우, WDATA64의 하위 32bit가 WDATA32로 네 번 전송된다음, WDATA64의 상위 32bit가 WDATA32로 네 번 전송되는 식으로 동작해야 한다.
 - WLAST 신호의 생성. 64 bit request가 Burst Length가 8 이상이어서 request가

두 번에 나누어 전송되었을 경우에는 WLAST 신호도 제 때를 맞추어 전송되어야 한다.

- 이런 처리를 위해서는 다음과 같은 정보가 필요하다.
 - Burst Length를 알아야 한다.
 - 64 bit request 였는지 detect할 필요가 있다.
 - address와 size를 알아야 한다.
- 또한 이런 모든 정보가 queuing되어야 한다. Write Data Interleaving은 지원하지 않을 것이므로 WDATA channel을 order에 맞추어 처리하면 된다. 따라서 정보는 FIFO 형식의 que에 저장되어도 무방하다.

3. Write Response Channel 처리

- 32bit interface에서 들어오는 모든 Write Response를 64bit interface로 전달해서는 안된다. 64 bit request가 split되었을 경우 Write response도 두번에 걸쳐서 들어오게 된다. 이 때는 첫번째 response의 결과를 latch해 두었다가 두 번째 response가 들어올 때 그것과 ORing하여 64bit interface에 전달해야 한다.
- 이것을 처리하기 위해서는 별도의 저장 공간에 request가 split되었는지 정보가 저장되어야 한다. 그 정보는 out-of-order completion으로 인해 queuing할 수는 없고, BID를 이용하여 lookup할 수 있는 random access 가능 register로 해야 한다.

(*) programmable parameter

- 64bit ONLY flag

64 bit request만 보내는 master를 위한 flag이다. 이 flag이 true이면 32bit 이하의 request를 처리할 필요가 없으므로 logic이 간결해 지게 된다.

- Error Detect On flag

보통의 64bit Master는 Memory에서 읽고 쓰는 일을 수행할 때 특별히 error 처리를 수행하지 않는다. 따라서 Write response channel를 가지고 있을 필요도 없다. 또한 Read Error에 대해서 처리할 필요도 없다. 이 경우에 대해서는 Read Response가 항상 OKAY가 되도록 assign하여 logic을 줄이게 된다.

- RID/WID 사용 flag

RID/WID를 사용하지 않으면 request는 모두 order에 맞추어 수행되므로 단순한 que의 형태로 request를 저장할 수 있게 된다.

- INCR ONLY flag

- Burst 종류별 flag

(*) 현재 설계되고 있는 Graphic Accelerator의 BUS 특성

- 64bit ONLY transaction
- Read Error detect 사용

- Write Error detect 사용(?)
- RID/WID 사용 안함
- INCR Burst만 사용함

(*) test 방법

기존의 test master를 64bit용으로 변경한 후에 32bit SRAM controller와 연결하여 테스트한다.